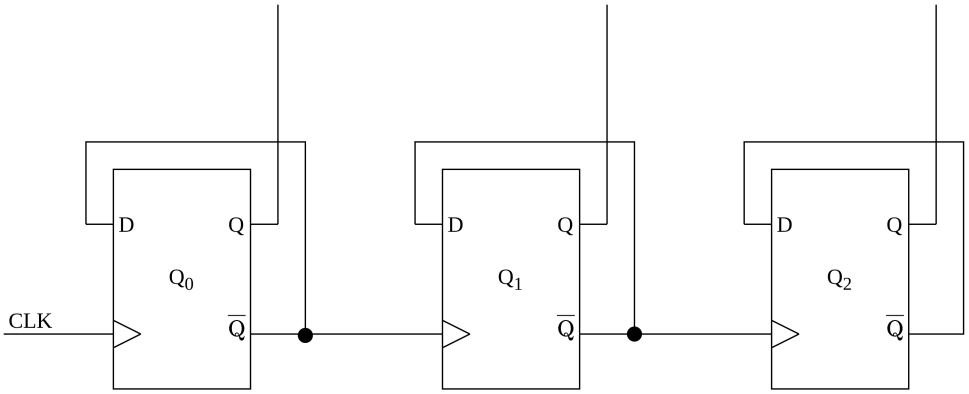
Sequential circuits

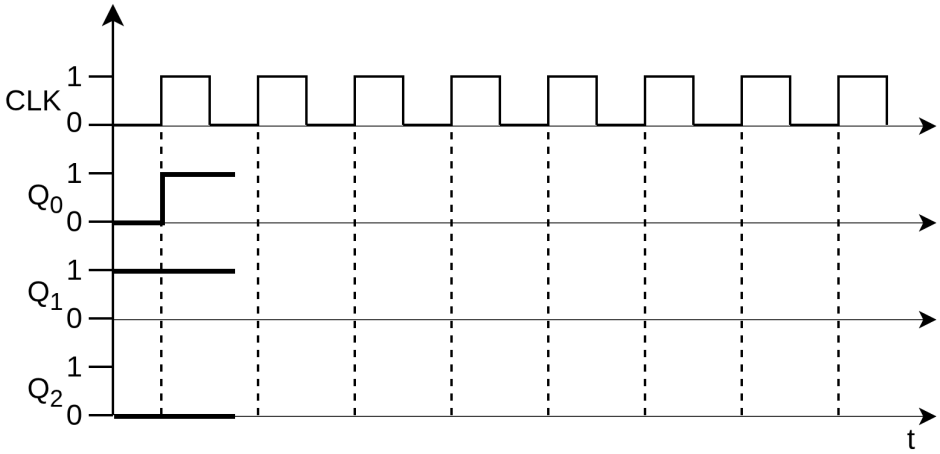
A 3-bit ripple counter is shown below:



(a) Relating to a rising edge on the `CLK` signal, describe exactly how this could cause a change of state of `Q1`.

> **Solution:** A rising edge on the `CLK` input signal will cause Q0 to store the value present on it's [D]ata input. If this is a transition from $1 \rightarrow 0$, then the value of $\overline{Q}$ will transition from $0 \rightarrow 1$, causing Q_1 to change state.

(b) On the axes below, complete the waveforms for Q0, Q1, Q2, and Q3 given the CLK input to the 3-bit ripple counter



(c) Considering the state change from `111` $\rightarrow$ `000`, explain why all bits do not change at the same time. In your answer, state whether the circuit is synchronous or asynchronous, and explain any intermediary states.

> **Solution:** Because each [D]ata signal is connected to its own $\overline{Q}$, every rising edge on any clock input will cause that flip flop to change state.

The rising edge on the `CLK` signal triggers a change of state of Q_0 only: `110`. Because Q_0 changes from a $1 \rightarrow 0$, $\overline{Q_0}$ changes from a $0 \rightarrow 1$, which Q_1 will see as a rising edge on its clock input, triggering a change of state of Q_1 so the overall system will be `100`. Similarly, the $1 \rightarrow 0$ of Q_1 will trigger a change of state of Q_2, finally triggering a change to `000`.

(d) Why is the behaviour explained in part (c) undesirable?

Solution: Because it is possible that the counter could be sampled at any stage between the transition from `111` to `000`, which could result in the counter state being any of `110`, `100`, or `000` as being the state after `111`.

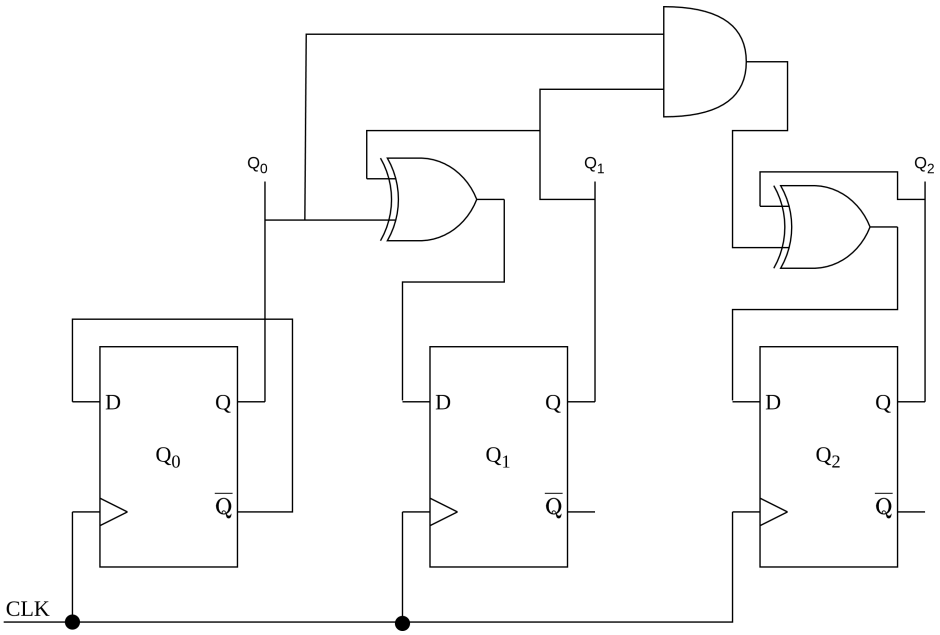
(e) We are going to fix the behaviour explained in part (c) by the addition of some combinational logic circuits, which will determine the next state of each bit for a 3-bit counter. Given the table below, complete the combinational logic design for $Q_{1,i+1}$ and $Q_{2,i+1}$, and draw the logic circuits in below.($Q_{0,i+1}$ is done with $\overline{Q_{0,i)}}$)

Current			Next		
$Q_{2,i}$	$Q_{1,i}$	$Q_{0,i}$	$Q_{2,i+1}$	$Q_{1,i+1}$	$Q_{0,i+1}$
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

Solution: Given $\oplus$ is the `XOR` operator:

$Q_{1,i+1} = Q_{1,i} \oplus Q_{0,i}$

$Q_{2,i+1} = Q_{2,i} \oplus (Q_{0,i}Q_{1,i})$



(f) For your solution, explain how you have solved the problem of asynchrony (i.e. explain when all of the flip flops change state from `111`, and what value is present on each flip flop's input, `D`). You can play with an equivalent circuit here.

Solution: Now every rising edge on the `CLK` input will cause each flip flop to read the value present on its own [D]ata input, and they will all change at the same time, so it is synchronous. The combinational logic circuits will ensure that each flip flop's [D]ata input will be set to the appropriate next state. For example, when the system state is $Q_0Q_1Q_2 = \texttt{111}$, $Q_{0,i+1} = \overline{Q_{0,i}} = \overline{1} = 0$, $Q_{1,i+1} = Q_{1,i} \oplus Q_{0,i} = 1 \oplus 1 = 0$, and $Q_{2,i+1} = Q_{2,i} \oplus (Q_{0,i}Q_{1,i}) = 1 \oplus (1 \cdot 1) = 1 \oplus 1 = 0$.

Therefore when the current states are `111`, the rising edge of the `CLK` input will cause all of the states to become `0` synchronously.

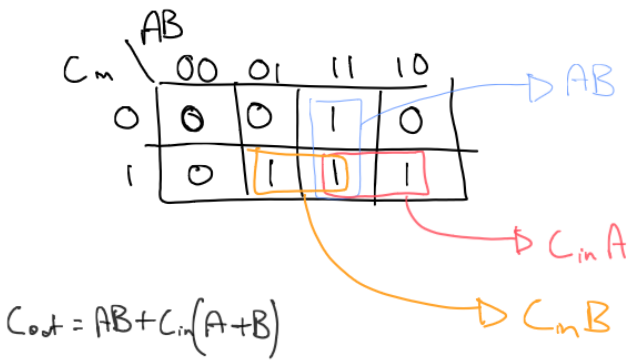
Lecture 5 - CPU

(a) Write an expression for a full adder's C_{out} in terms of the inputs A, B, and C_{in}.

Solution:

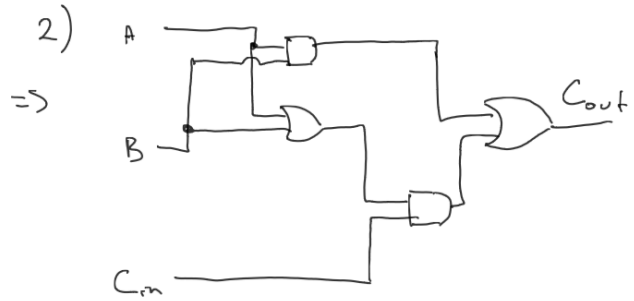
We know that the carry out will be true if two or more of the inputs are true:

C_{in}	A	B	C_{out}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



(b) Draw a circuit using AND, OR, and NOT logic gates that implements your expression for C_{out}.

Solution:

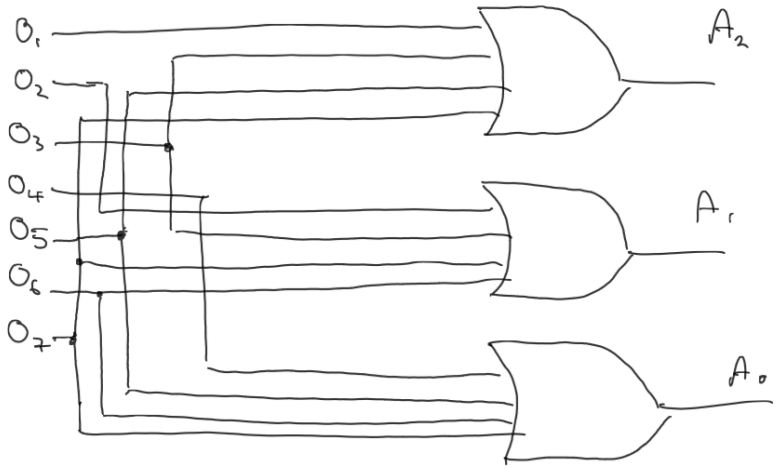


Lecture 6 - Combinational circuits

(a) What would the logic circuit for an 8:3 encoder look like? You may use 4-way `OR` gates in your answer.

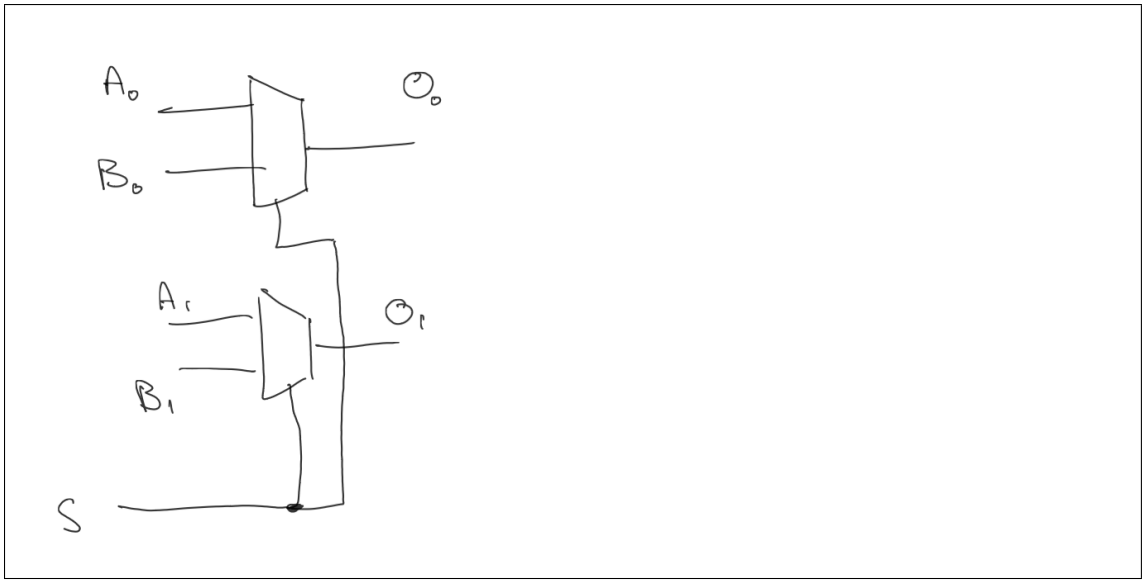
Solution:

$$A_2 = O_1 + O_3 + O_5 + O_7$$
$$A_1 = O_2 + O_3 + O_6 + O_7$$
$$A_0 = O_4 + O_5 + O_6 + O_7$$



(b) Implement a 4:2 mux using two 2:1 muxes.

Solution:

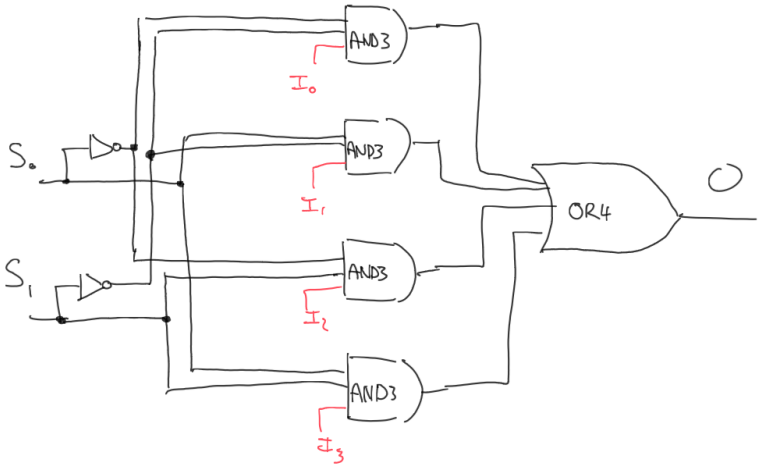


(c) Carry out the design process from a 4:1 multiplexer. You may use 3-way `AND` gates in your answer.

Solution:

S_1	S_0	I_0	I_1	I_2	I_3	O
0	0	0	x	x	x	0
0	0	1	x	x	x	1
0	1	x	0	x	x	0
0	1	x	1	x	x	1
1	0	x	x	0	x	0
1	0	x	x	1	x	1
1	1	x	x	x	0	0
1	1	x	x	x	1	1

⇒ we use AND 3s with the specific minterm for $S_0 S_1$, AND the appropriate I_n, then OR all the outputs of the AND-3



GPIO

Figure 1 is a simplified circuit diagram for a GPIO pin on the ATmega32U2 microcontroller.

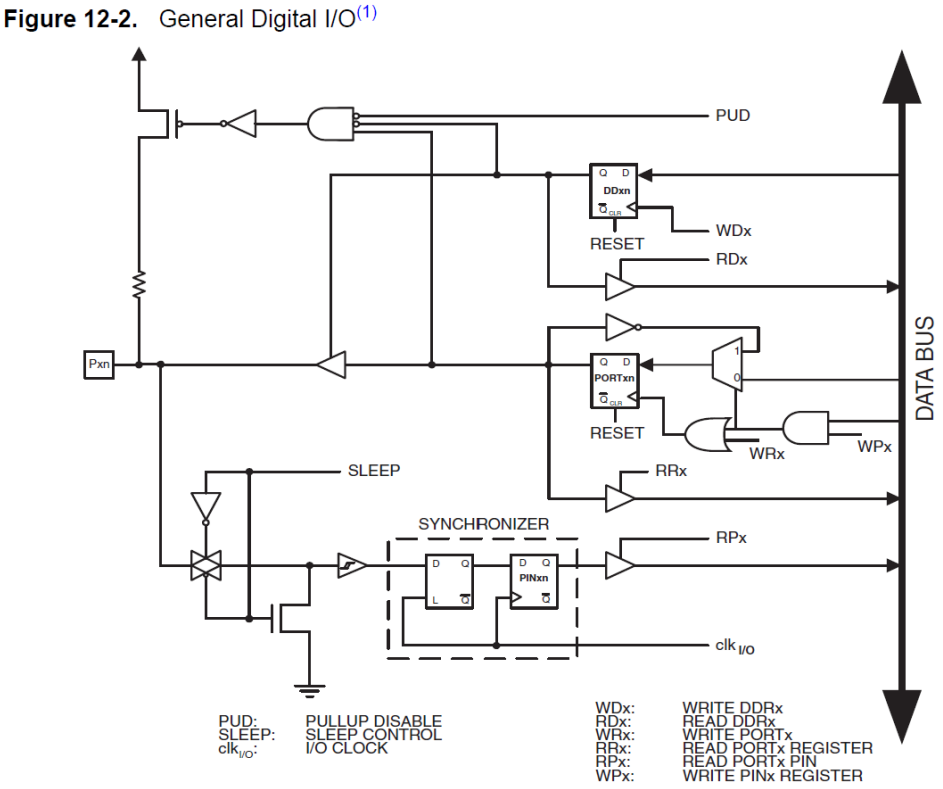


Figure 1: The logical circuit for one GPIO pin on the ATmega32U2 MCU.

(a) The `PUD` signal is a "Pull-Up Disable". Briefly explain why setting PUD to 1 would disable the pull-up resistor connected to *Pxn*.

Solution: From the diagram, we can see that the pull-up resistor is controlled by a p-channel MOSFET. Looking at the connection between PUD and the p-channel MOSFET we can see that:

- The p-channel MOSFET will be *on* when its gate voltage is *low*.
- The gate voltage of the MOSFET is low when the input to the inverter is *high*.
- The PUD input to the AND gate is inverted, so the AND output can only be high when PUD is *low*.

Thus setting PUD to 1 causes the output of the AND to be 0, which causes the inverter to output 1, which causes the p-channel MOSFET to be turned off.

(b) What is the boolean expression for enabling the pull-up resistor?

Solution: Per above, the output of the 3-way `AND` gate will need to be `HIGH`/1. Therefore, all three boolean variables will need to be `HIGH`/1 also, but two of the signals are inverted.

Therefore, the overall expression needed is: $\text{ENABLE}_{\text{PULLUP}} = \overline{\texttt{PUD}} \cdot \overline{\texttt{DDxn}} \cdot \texttt{PORTnx}$

(c) Under what conditions would the tristate buffer between *Pxn* and the *PORTxn* flip-flop be turned ON, i.e. conducting?

Solution: The tristate buffer will be ON when the Q output of the *DDxn* flip-flop is 1, i.e., when a 1 is written to that bit of the Data Direction Register.

(d) What values need to stored in the DDxn and PORTxn flip flops to output a digital `HIGH` signal on the pin?

Solution: As above, the *DDxn* flipflop will need to be set to 1/`HIGH` in order to activate the tristate buffer. Doing so allows the tristate buffer to conduct, therefore meaning that whatever digital signal is coming in to the tristate buffer will be sent through to the pin. To have this signal `HIGH`, the *PORTxn* register will also need to be set to 1/`HIGH`

(e) (Extra) What is the role played by the 2:1 multiplexer that provides the input to the *PORTxn* flip-flop?

Solution: The multiplexer sends one of two possible signals to the input of the *PORTxn* flip-flop:

- The inverted value of the current *PORTxn* output
- A value from the data bus

Therefore, it allows easy toggling of the specific bit to turn it from off to on without having to read the current value of the register.

Learning about computers using computers

Complete up to the multi-bit adder in the Nandgame (https://nandgame.com/).

END OF QUESTIONS